Plagiarism Detection Visualization

Nate Phillips
Josh Crowson

Purpose

- The purpose of this project was to design a system that could detect plagiarism and display the results in an intuitive human-friendly fashion.

Two Parts

- A system to programmatically detect any instances of plagiarism and export the plagiarized words to an output file.
- A system to take the words from the output file, as well as the two text files, and visualize the plagiarism level of the document.

Plagiarism Detection Software

- The plagiarism detection program takes in two input text files and individually breaks the the texts into word groups of size 3-7.
- When it breaks up the text, it strips out all punctuation.

Plagiarism Detection Software

- The program then compares the word groupings from one text to the word groupings of the other text and stores them in another list.
- Then the program reduces the list to remove redundant word groupings.

Plagiarism Detection Software

- Primary text: 8 pages
- Secondary text: 5 pages
- Word groupings of in the primary text: 11040
- Word groupings in the secondary text: 5345
- 80 word groupings in common between the two sets
- The 80 word groupings is reduced to 12 unique word groupings

Plagiarism Detection Software

80 word groupings: [[the, c, programming], [c, programming, language], [one, of, the], [one, of, the], [the, c, programming], [c, programming, language], [the, b, language], [ritchie, modified, b], [ritchie, modified, b], [modified, b, which], [b, which, eventually], [which, eventually, became], [the, c, programming], [c, programming, language], [as, well, as], [ken, thompson, developed], [thompson, developed, the], [developed, the, c], [the, c, programming], [c, programming, language], [with, dennis, ritchie], [with, dennis, ritchie], [when, it, was], [dennis, ritchie, dennis], [ritchie, dennis, ritchie], [developed, the, c], [the, c, programming], [c, programming, language], [the, c, programming], [c, programming, language], [the, c, programming], [c, programming, language], [with, dennis, ritchie], [with, dennis, ritchie], [the, c, language], [the, c, language], [the, b, language], [of, the, c], [the, c, programming], [c, programming, language], [as, well, as], [part, of, the], [of, the, software], [the, computer, science], [the, computer, science], [is, based, on], [the, c, programming, language], [the, c, programming, language], [ritchie, modified, b, which], [ritchie, modified, b, which], [modified, b, which, eventually], [b, which, eventually, became], [the, c, programming, language], [thompson, ken, thompson, developed], [ken, thompson, developed, the], [thompson, developed, the, c], [developed, the, c, programming], [the, c, programming, language], [dennis, ritchie, dennis, ritchie], [developed, the, c, programming], [the, c, programming, language], [the, c, programming, language], [the, c, programming, language], [the, c, programming, language], [the, c, programming, language, developed], [ritchie, modified, b, which, eventually], [ritchie, modified, b, which, eventually], [modified, b, which, eventually, became], [thompson, ken, thompson, developed, the], [ken, thompson, developed, the, c], [thompson, developed, the, c, programming], [developed, the, c, programming, language], [developed, the, c, programming, language], [ritchie, modified, b, which, eventually, became], [ritchie, modified, b, which, eventually, became], [thompson, ken, thompson, developed, the, c], [ken, thompson, developed, the, c, programming], [thompson, developed, the, c, programming, language], [thompson, ken, thompson, developed, the, c, programming], [ken, thompson, developed, the, c, programming, language]]

Plagiarism Detection Software

Reduced to 12 word groupings:

[[one, of, the], [the, b, language], [as, well, as], [with, dennis, ritchie], [when, it, was], [of, the, c], [part, of, the], [of, the, software], [the, computer, science], [is, based, on], [ritchie, modified, b, which, eventually, became], [ken, thompson, developed, the, c, programming, language]]

Plagiarism Detection Software

Example of the reduction:

[ritchie, modified, b]
[modified, b, which]
[b, which, eventually]
[which, eventually, became]
[ritchie, modified, b, which]
[modified, b, which, eventually]
[b, which, eventually, became]
[ritchie, modified, b, which, eventually, became]

Becomes:

[ritchie, modified, b, which, eventually, became]

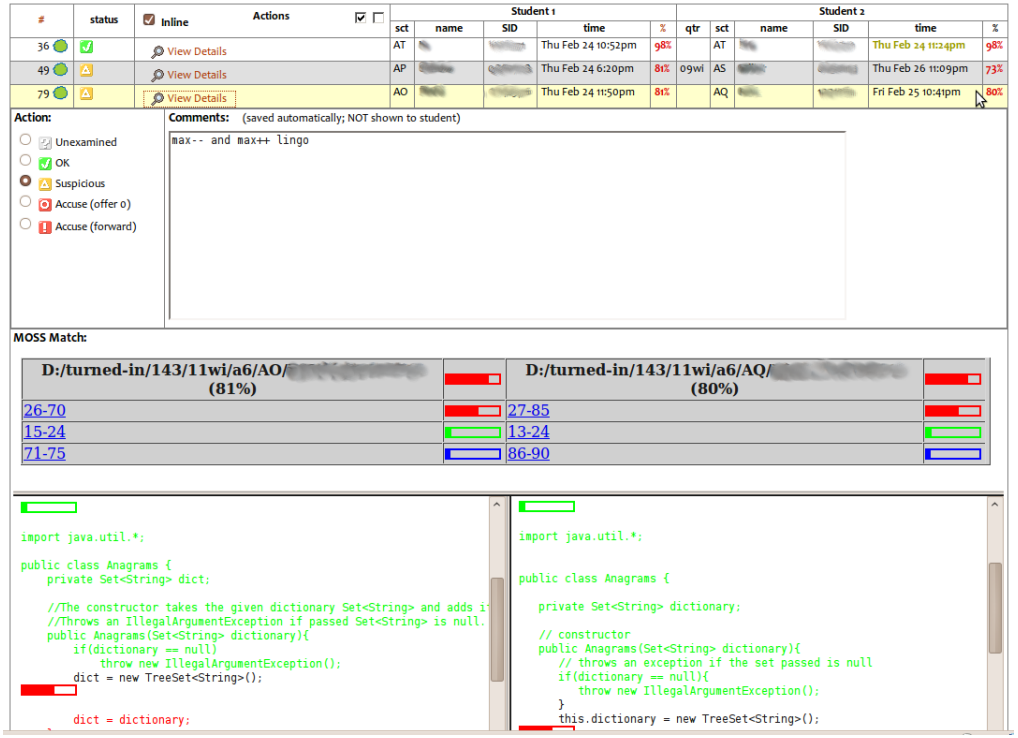
Improvements

- A dictionary of synonyms
- Take into consideration the lengths of the words
- Handle punctuation better

Plagiarism Visualization

- Plagiarism detection is an important field with definite practical significance. However, while some work has been done, by and large we have made no major advances in displaying plagiarism in the past five or six years.

Moss Example



#	status	Inline	Actions	Student 1 sct	Student 1 name	Student 1 SID	Student 1 time	Student 1 %	qtr	Student 2 sct	Student 2 name	Student 2 SID	Student 2 time	Student 2 %
36			View Details	AT	[redacted]	[redacted]	Thu Feb 24 10:52pm	98%		AT	[redacted]	[redacted]	Thu Feb 24 11:24pm	98%
49			View Details	AP	[redacted]	[redacted]	Thu Feb 24 6:20pm	81%	09wi	AS	[redacted]	[redacted]	Thu Feb 26 11:09pm	73%
79			View Details	AO	[redacted]	[redacted]	Thu Feb 24 11:50pm	81%		AQ	[redacted]	[redacted]	Fri Feb 25 10:41pm	80%

Action:
- Unexamined
- OK
- Suspicious
- Accuse (offer 0)
- Accuse (forward)

Comments: (saved automatically; NOT shown to student)

```
max-- and max++ lingo
```

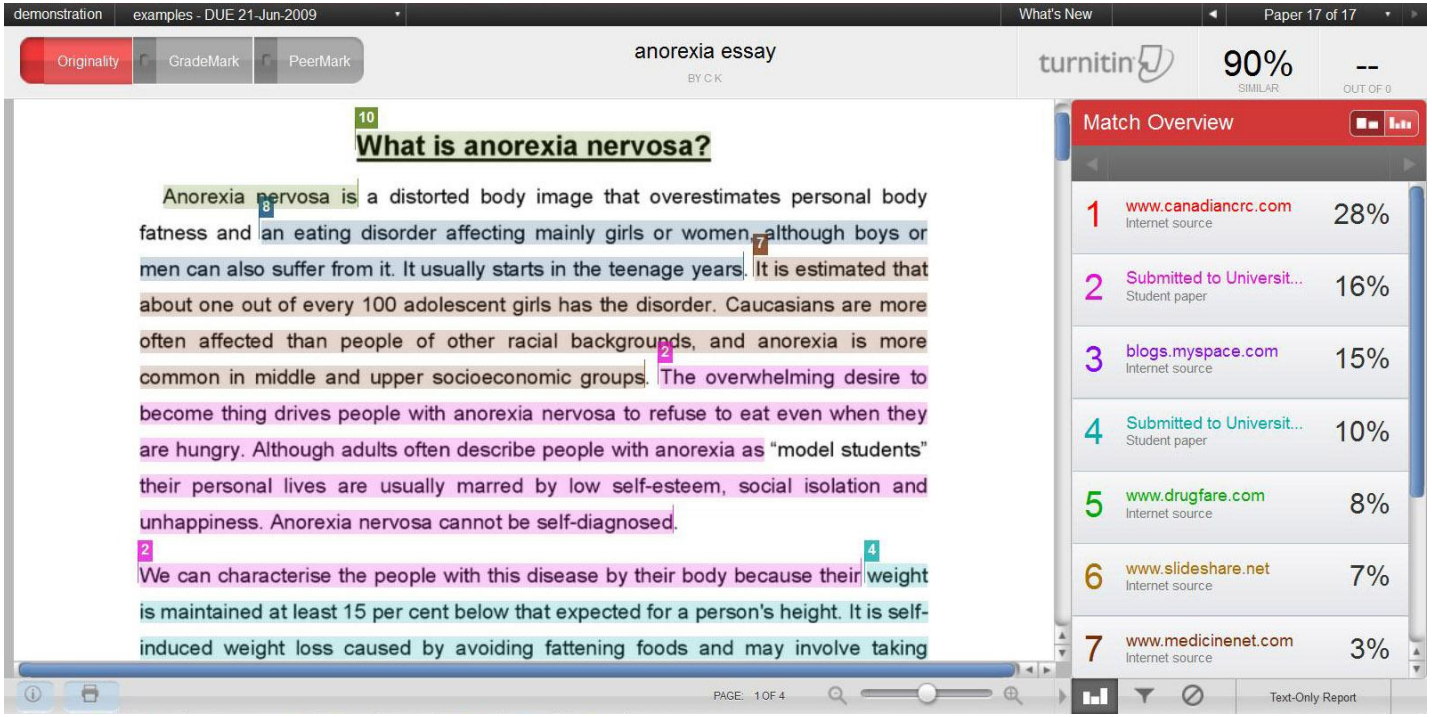
MOSS Match:

D:/turned-in/143/11wi/a6/AO/[redacted] (81%)	D:/turned-in/143/11wi/a6/AQ/[redacted] (80%)
26-70	27-85
15-24	13-24
71-75	86-90

```
import java.util.*;

public class Anagrams {
    private Set<String> dict;

    //The constructor takes the given dictionary Set<String> and adds i
    //Throws an IllegalArgumentException if passed Set<String> is null.
    public Anagrams(Set<String> dictionary){
        if(dictionary == null)
            throw new IllegalArgumentException();
        dict = new TreeSet<String>();

        dict = dictionary;
```

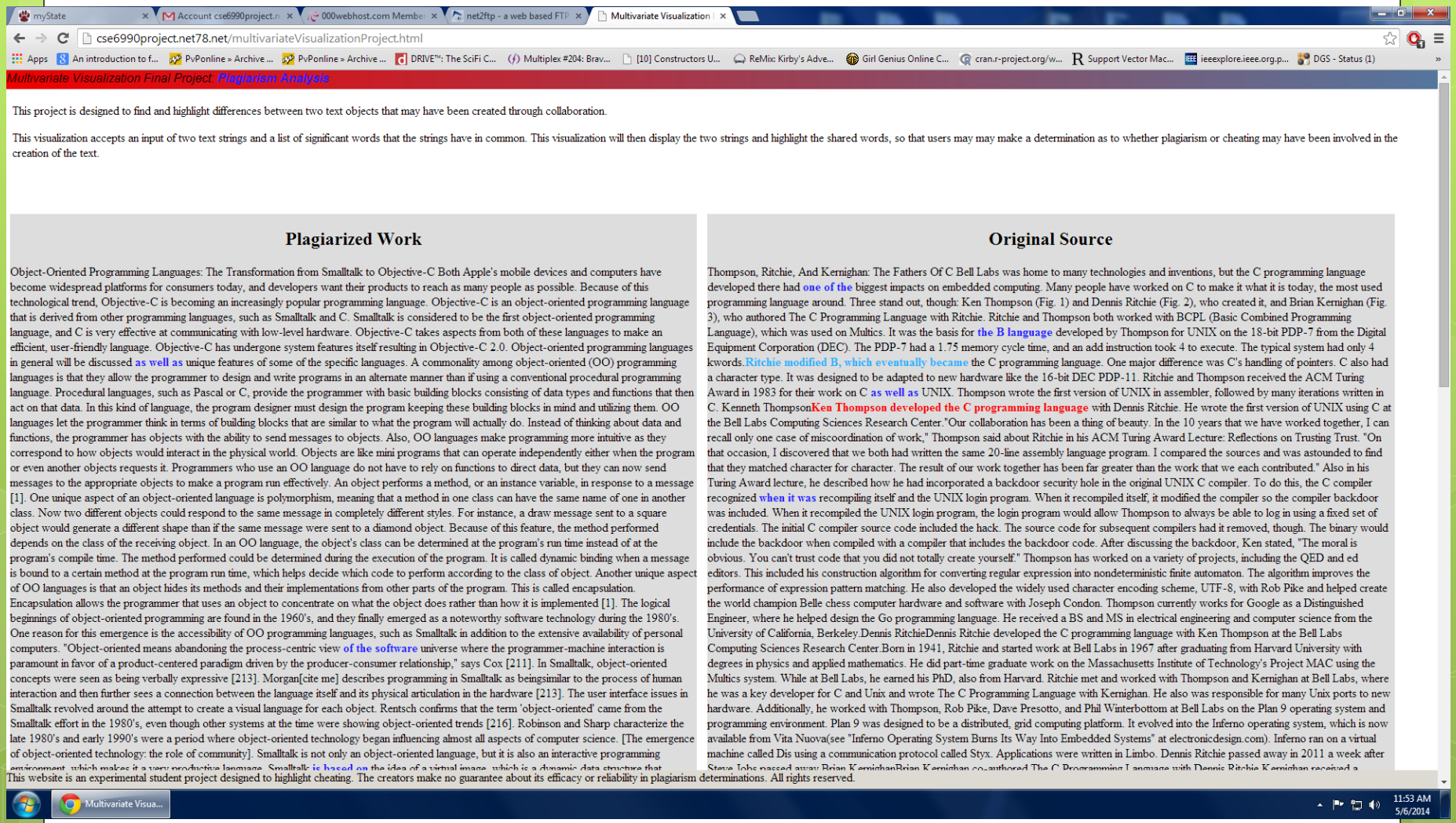
```
import java.util.*;


public class Anagrams {

   private Set<String> dictionary;

   // constructor
   public Anagrams(Set<String> dictionary){
      // throws an exception if the set passed is null
      if(dictionary == null){
          throw new IllegalArgumentException();
      }
      this.dictionary = new TreeSet<String>();
```

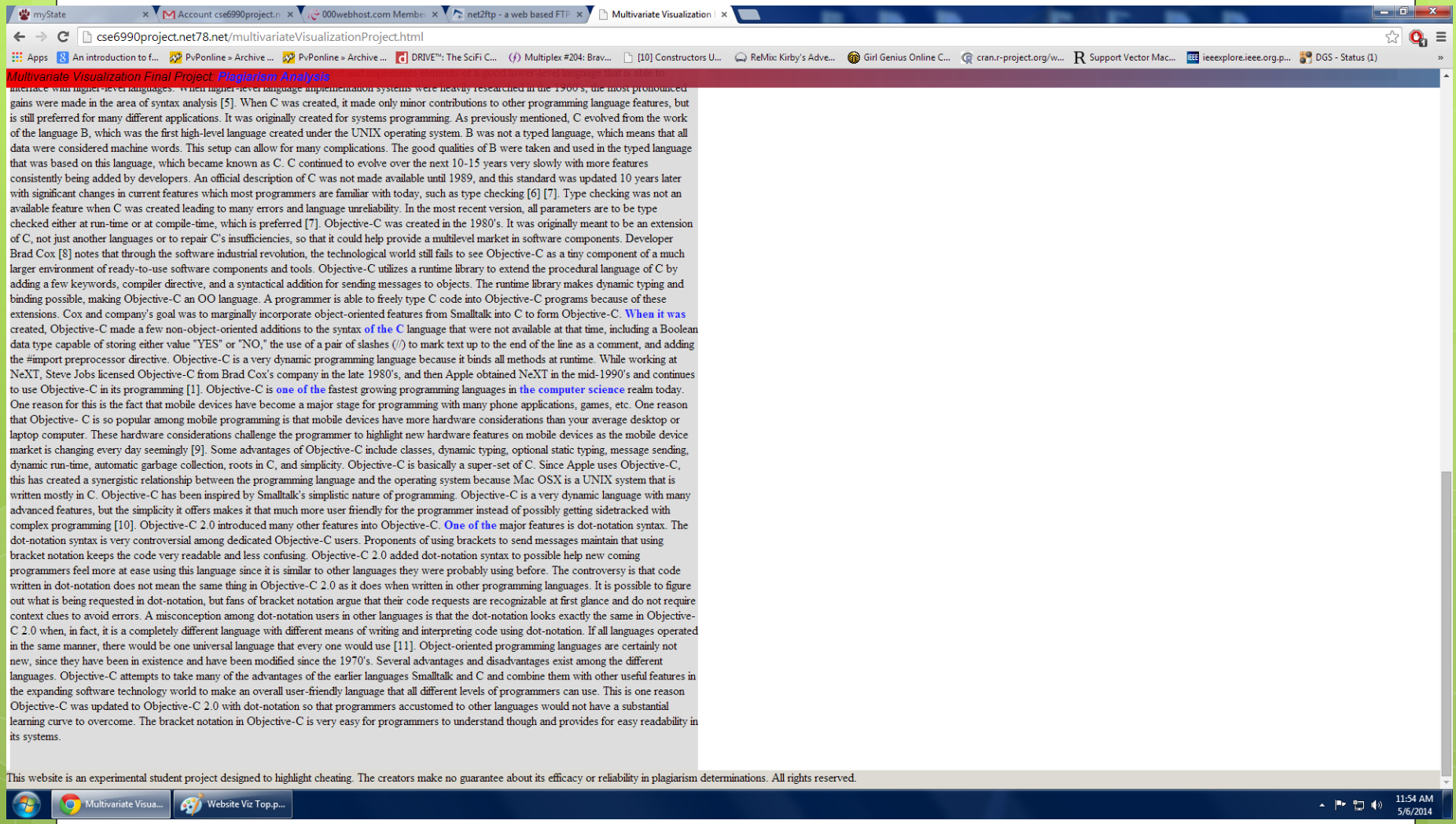
Turnitin.com Example



Our visualization

- For our visualization, we did something similar, though less in-depth, than the examples listed previously. We went through and highlighted any plagiarized words in both documents.


myState
Account cse6990project.n
000webhost.com Member
net2ftp - a web based FTP
Multivariate Visualization
cse6990project.net78.net/multivariateVisualizationProject.html
Apps
An introduction to f...
PyPonline » Archive ...
PyPonline » Archive ...
DRIVE™: The SciFi C...
Multiplex #204: Brav...
[10] Constructors U...
ReMix: Kirby's Adve...
Girl Genius Online C...
cran.r-project.org/w...
Support Vector Mac...
ieeexplore.ieee.org.p...
DGS - Status (1)
Multivariate Visualization Final Project: Plagiarism Analysis
This project is designed to find and highlight differences between two text objects that may have been created through collaboration.
This visualization accepts an input of two text strings and a list of significant words that the strings have in common. This visualization will then display the two strings and highlight the shared words, so that users may may make a determination as to whether plagiarism or cheating may have been involved in the creation of the text.
Plagiarized Work
Object-Oriented Programming Languages: The Transformation from Smalltalk to Objective-C Both Apple's mobile devices and computers have become widespread platforms for consumers today, and developers want their products to reach as many people as possible. Because of this technological trend, Objective-C is becoming an increasingly popular programming language. Objective-C is an object-oriented programming language that is derived from other programming languages, such as Smalltalk and C. Smalltalk is considered to be the first object-oriented programming language, and C is very effective at communicating with low-level hardware. Objective-C takes aspects from both of these languages to make an efficient, user-friendly language. Objective-C has undergone system features itself resulting in Objective-C 2.0. Object-oriented programming languages in general will be discussed as well as unique features of some of the specific languages. A commonality among object-oriented (OO) programming languages is that they allow the programmer to design and write programs in an alternate manner than if using a conventional procedural programming language. Procedural languages, such as Pascal or C, provide the programmer with basic building blocks consisting of data types and functions that then act on that data. In this kind of language, the program designer must design the program keeping these building blocks in mind and utilizing them. OO languages let the programmer think in terms of building blocks that are similar to what the program will actually do. Instead of thinking about data and functions, the programmer has objects with the ability to send messages to objects. Also, OO languages make programming more intuitive as they correspond to how objects would interact in the physical world. Objects are like mini programs that can operate independently either when the program or even another objects requests it. Programmers who use an OO language do not have to rely on functions to direct data, but they can now send messages to the appropriate objects to make a program run effectively. An object performs a method, or an instance variable, in response to a message [1]. One unique aspect of an object-oriented language is polymorphism, meaning that a method in one class can have the same name of one in another class. Now two different objects could respond to the same message in completely different styles. For instance, a draw message sent to a square object would generate a different shape than if the same message were sent to a diamond object. Because of this feature, the method performed depends on the class of the receiving object. In an OO language, the object's class can be determined at the program's run time instead of at the program's compile time. The method performed could be determined during the execution of the program. It is called dynamic binding when a message is bound to a certain method at the program run time, which helps decide which code to perform according to the class of object. Another unique aspect of OO languages is that an object hides its methods and their implementations from other parts of the program. This is called encapsulation. Encapsulation allows the programmer that uses an object to concentrate on what the object does rather than how it is implemented [1]. The logical beginnings of object-oriented programming are found in the 1960's, and they finally emerged as a noteworthy software technology during the 1980's. One reason for this emergence is the accessibility of OO programming languages, such as Smalltalk in addition to the extensive availability of personal computers. "Object-oriented means abandoning the process-centric view of the software universe where the programmer-machine interaction is paramount in favor of a product-centered paradigm driven by the producer-consumer relationship," says Cox [211]. In Smalltalk, object-oriented concepts were seen as being verbally expressive [213]. Morgan[cite me] describes programming in Smalltalk as beingsimilar to the process of human interaction and then further sees a connection between the language itself and its physical articulation in the hardware [213]. The user interface issues in Smalltalk revolved around the attempt to create a visual language for each object. Rentsch confirms that the term 'object-oriented' came from the Smalltalk effort in the 1980's, even though other systems at the time were showing object-oriented trends [216]. Robinson and Sharp characterize the late 1980's and early 1990's were a period where object-oriented technology began influencing almost all aspects of computer science. [The emergence of object-oriented technology: the role of community]. Smalltalk is not only an object-oriented language, but it is also an interactive programming environment, which makes it a very productive language. Smalltalk is based on the idea of a virtual image, which is a dynamic data structure that
Original Source
Thompson, Ritchie, And Kernighan: The Fathers Of C Bell Labs was home to many technologies and inventions, but the C programming language developed there had one of the biggest impacts on embedded computing. Many people have worked on C to make it what it is today, the most used programming language around. Three stand out, though: Ken Thompson (Fig. 1) and Dennis Ritchie (Fig. 2), who created it, and Brian Kernighan (Fig. 3), who authored The C Programming Language with Ritchie. Ritchie and Thompson both worked with BCPL (Basic Combined Programming Language), which was used on Multics. It was the basis for the B language developed by Thompson for UNIX on the 18-bit PDP-7 from the Digital Equipment Corporation (DEC). The PDP-7 had a 1.75 memory cycle time, and an add instruction took 4 to execute. The typical system had only 4 kwords.Ritchie modified B, which eventually became the C programming language. One major difference was C's handling of pointers. C also had a character type. It was designed to be adapted to new hardware like the 16-bit DEC PDP-11. Ritchie and Thompson received the ACM Turing Award in 1983 for their work on C as well as UNIX. Thompson wrote the first version of UNIX in assembler, followed by many iterations written in C. Kenneth ThompsonKen Thompson developed the C programming language with Dennis Ritchie. He wrote the first version of UNIX using C at the Bell Labs Computing Sciences Research Center."Our collaboration has been a thing of beauty. In the 10 years that we have worked together, I can recall only one case of miscoordination of work," Thompson said about Ritchie in his ACM Turing Award Lecture: Reflections on Trusting Trust. "On that occasion, I discovered that we both had written the same 20-line assembly language program. I compared the sources and was astounded to find that they matched character for character. The result of our work together has been far greater than the work that we each contributed." Also in his Turing Award lecture, he described how he had incorporated a backdoor security hole in the original UNIX C compiler. To do this, the C compiler recognized when it was recompiling itself and the UNIX login program. When it recompiled itself, it modified the compiler so the compiler backdoor was included. When it recompiled the UNIX login program, the login program would allow Thompson to always be able to log in using a fixed set of credentials. The initial C compiler source code included the hack. The source code for subsequent compilers had it removed, though. The binary would include the backdoor when compiled with a compiler that includes the backdoor code. After discussing the backdoor, Ken stated, "The moral is obvious. You can't trust code that you did not totally create yourself." Thompson has worked on a variety of projects, including the QED and ed editors. This included his construction algorithm for converting regular expression into nondeterministic finite automaton. The algorithm improves the performance of expression pattern matching. He also developed the widely used character encoding scheme, UTF-8, with Rob Pike and helped create the world champion Belle chess computer hardware and software with Joseph Condon. Thompson currently works for Google as a Distinguished Engineer, where he helped design the Go programming language. He received a BS and MS in electrical engineering and computer science from the University of California, Berkeley.Dennis RitchieDennis Ritchie developed the C programming language with Ken Thompson at the Bell Labs Computing Sciences Research Center.Born in 1941, Ritchie and started work at Bell Labs in 1967 after graduating from Harvard University with degrees in physics and applied mathematics. He did part-time graduate work on the Massachusetts Institute of Technology's Project MAC using the Multics system. While at Bell Labs, he earned his PhD, also from Harvard. Ritchie met and worked with Thompson and Kernighan at Bell Labs, where he was a key developer for C and Unix and wrote The C Programming Language with Kernighan. He also was responsible for many Unix ports to new hardware. Additionally, he worked with Thompson, Rob Pike, Dave Presotto, and Phil Winterbottom at Bell Labs on the Plan 9 operating system and programming environment. Plan 9 was designed to be a distributed, grid computing platform. It evolved into the Inferno operating system, which is now available from Vita Nuova(see "Inferno Operating System Burns Its Way Into Embedded Systems" at electronicdesign.com). Inferno ran on a virtual machine called Dis using a communication protocol called Styx. Applications were written in Limbo. Dennis Ritchie passed away in 2011 a week after Steve Jobs passed away.Brian KernighanBrian Kernighan co-authored The C Programming Language with Dennis Ritchie.Kernighan received a
This website is an experimental student project designed to highlight cheating. The creators make no guarantee about its efficacy or reliability in plagiarism determinations. All rights reserved.
Multivariate Visua...
11:53 AM
5/6/2014


myState
Account cse6990project.n
000webhost.com Member
net2ftp - a web based FTP
Multivariate Visualization
cse6990project.net78.net/multivariateVisualizationProject.html
Apps
An introduction to f...
PvPonline » Archive ...
PvPonline » Archive ...
DRIVE™: The SciFi C...
Multiplex #204: Brav...
[10] Constructors U...
ReMix: Kirby's Adve...
Girl Genius Online C...
cran.r-project.org/w...
Support Vector Mac...
ieeexplore.ieee.org.p...
DGS - Status (1)
Multivariate Visualization Final Project Plagiarism Analysis
interface with higher-level languages. When higher-level language implementation systems were heavily researched in the 1960's, the most pronounced gains were made in the area of syntax analysis [5]. When C was created, it made only minor contributions to other programming language features, but is still preferred for many different applications. It was originally created for systems programming. As previously mentioned, C evolved from the work of the language B, which was the first high-level language created under the UNIX operating system. B was not a typed language, which means that all data were considered machine words. This setup can allow for many complications. The good qualities of B were taken and used in the typed language that was based on this language, which became known as C. C continued to evolve over the next 10-15 years very slowly with more features consistently being added by developers. An official description of C was not made available until 1989, and this standard was updated 10 years later with significant changes in current features which most programmers are familiar with today, such as type checking [6] [7]. Type checking was not an available feature when C was created leading to many errors and language unreliability. In the most recent version, all parameters are to be type checked either at run-time or at compile-time, which is preferred [7]. Objective-C was created in the 1980's. It was originally meant to be an extension of C, not just another languages or to repair C's insufficiencies, so that it could help provide a multilevel market in software components. Developer Brad Cox [8] notes that through the software industrial revolution, the technological world still fails to see Objective-C as a tiny component of a much larger environment of ready-to-use software components and tools. Objective-C utilizes a runtime library to extend the procedural language of C by adding a few keywords, compiler directive, and a syntactical addition for sending messages to objects. The runtime library makes dynamic typing and binding possible, making Objective-C an OO language. A programmer is able to freely type C code into Objective-C programs because of these extensions. Cox and company's goal was to marginally incorporate object-oriented features from Smalltalk into C to form Objective-C. When it was created, Objective-C made a few non-object-oriented additions to the syntax of the C language that were not available at that time, including a Boolean data type capable of storing either value "YES" or "NO," the use of a pair of slashes (//) to mark text up to the end of the line as a comment, and adding the #import preprocessor directive. Objective-C is a very dynamic programming language because it binds all methods at runtime. While working at NeXT, Steve Jobs licensed Objective-C from Brad Cox's company in the late 1980's, and then Apple obtained NeXT in the mid-1990's and continues to use Objective-C in its programming [1]. Objective-C is one of the fastest growing programming languages in the computer science realm today. One reason for this is the fact that mobile devices have become a major stage for programming with many phone applications, games, etc. One reason that Objective- C is so popular among mobile programming is that mobile devices have more hardware considerations than your average desktop or laptop computer. These hardware considerations challenge the programmer to highlight new hardware features on mobile devices as the mobile device market is changing every day seemingly [9]. Some advantages of Objective-C include classes, dynamic typing, optional static typing, message sending, dynamic run-time, automatic garbage collection, roots in C, and simplicity. Objective-C is basically a super-set of C. Since Apple uses Objective-C, this has created a synergistic relationship between the programming language and the operating system because Mac OSX is a UNIX system that is written mostly in C. Objective-C has been inspired by Smalltalk's simplistic nature of programming. Objective-C is a very dynamic language with many advanced features, but the simplicity it offers makes it that much more user friendly for the programmer instead of possibly getting sidetracked with complex programming [10]. Objective-C 2.0 introduced many other features into Objective-C. One of the major features is dot-notation syntax. The dot-notation syntax is very controversial among dedicated Objective-C users. Proponents of using brackets to send messages maintain that using bracket notation keeps the code very readable and less confusing. Objective-C 2.0 added dot-notation syntax to possible help new coming programmers feel more at ease using this language since it is similar to other languages they were probably using before. The controversy is that code written in dot-notation does not mean the same thing in Objective-C 2.0 as it does when written in other programming languages. It is possible to figure out what is being requested in dot-notation, but fans of bracket notation argue that their code requests are recognizable at first glance and do not require context clues to avoid errors. A misconception among dot-notation users in other languages is that the dot-notation looks exactly the same in Objective-C 2.0 when, in fact, it is a completely different language with different means of writing and interpreting code using dot-notation. If all languages operated in the same manner, there would be one universal language that every one would use [11]. Object-oriented programming languages are certainly not new, since they have been in existence and have been modified since the 1970's. Several advantages and disadvantages exist among the different languages. Objective-C attempts to take many of the advantages of the earlier languages Smalltalk and C and combine them with other useful features in the expanding software technology world to make an overall user-friendly language that all different levels of programmers can use. This is one reason Objective-C was updated to Objective-C 2.0 with dot-notation so that programmers accustomed to other languages would not have a substantial learning curve to overcome. The bracket notation in Objective-C is very easy for programmers to understand though and provides for easy readability in its systems.
This website is an experimental student project designed to highlight cheating. The creators make no guarantee about its efficacy or reliability in plagiarism determinations. All rights reserved.
Multivariate Visua...
Website Viz Top.p...
11:54 AM
5/6/2014

Possible Improvements

- We considered a couple of alternate approaches/additional features as well.
  - Arc Graphs
  - Highlighting
  - Some sort of analytical analysis

Questions?

Sources

- Turnitin.com example: https://www.google.com/url?sa=i&rct=j&q=&esrc=s&source=images&cd=&cad=rja&uact=8&docid=mxaBJqub061jBM&tbnid=U2TnTBtvZdjLnM:&ved=0CAUQjRw&url=https%3A%2F%2Fwww.wlv.ac.uk%2Fdefault.aspx%3Fpage%3D32434&ei=ChdpU9WLNaPuyAGZo4C4Dw&bvm=bv.65788261,d.aWc&psig=AFQjCNEqa_3NBYzyCT-uVRSkAI3XkWpVqw&ust=1399482500059273